U.E 5.1 - Intelligence Artificielle : Machine learning
Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France

BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr
BENIGNO DA SILVA Matheus Henrique
matheus.benigno@ensta.fr

Sommaire

1 | 3 Examen des données, prétraitements et extraction des descripteurs 3
1.1 | 3.1 Question . . . 3
1.2 | 3.2 Question . . . 4
1.3 | 3.3 Question . . . 5
1.4 | 3.4 Question . . . 5

2 | 4 Constitution d'un ensemble de données d'apprentissage 7
2.1 | 4.1 Question . . . 7

3 | 5 Première analyse des données 8
3.1 | 5.1 Question . . . 8
3.2 | 5.2 Question . . . 10

4 | 6 Approche non supervisée par la méthode des k-means 10
4.1 | 6.1 Question . . . 10
4.2 | 6.2 Question . . . 10
4.3 | 6.3 Question . . . 12
4.4 | 6.4 Question . . . 13
4.5 | 6.5 Question . . . 17
4.6 | 6.6 Question . . . 18
4.7 | 6.7 Question . . . 20
4.8 | 6.8 Question . . . 21

5 | 7 Approche non-supervisée par GMM **21**
5.1 | 7.1 Question 21
5.2 | 7.2 Question 22
5.3 | 7.3 Question 26
5.4 | 7.4 Question 27

1 | 3 Examen des données, prétraitements et extraction des descripteurs

1.1 | 3.1 Question

```python
#%% package
import numpy as np
import matplotlib.pyplot as plt

from loadImages import loadImages
from selectFeatureVectors import selectFeatureVectors
from displayFeatures2d import displayFeatures2d
from displayFeatures3d import displayFeatures3d

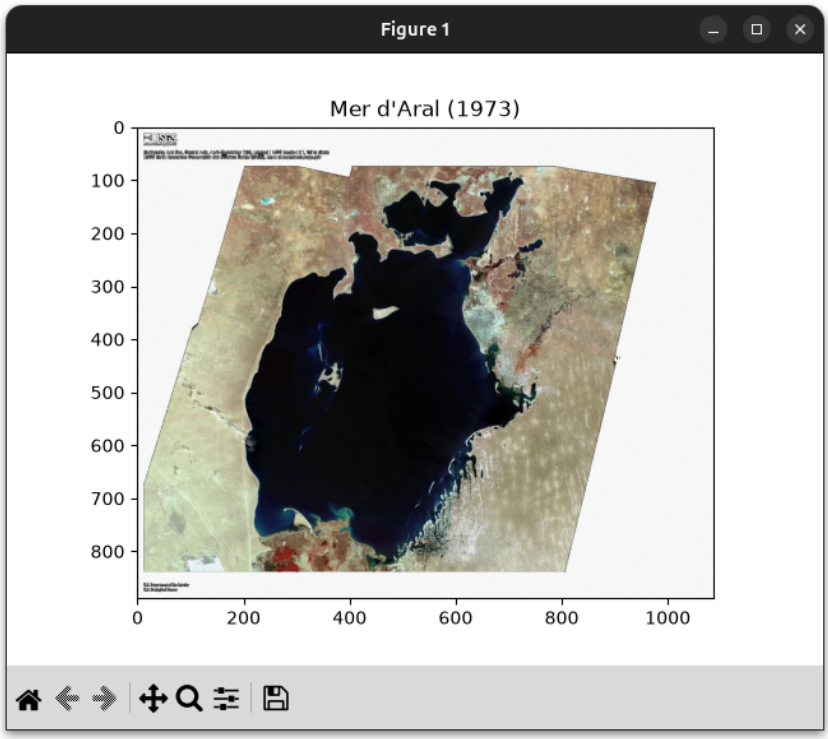

#%% def preprocessing
def preprocessing():
    # ------------------------------------------------
    # YOUR CODE HERE
    img73, img87 = loadImages()

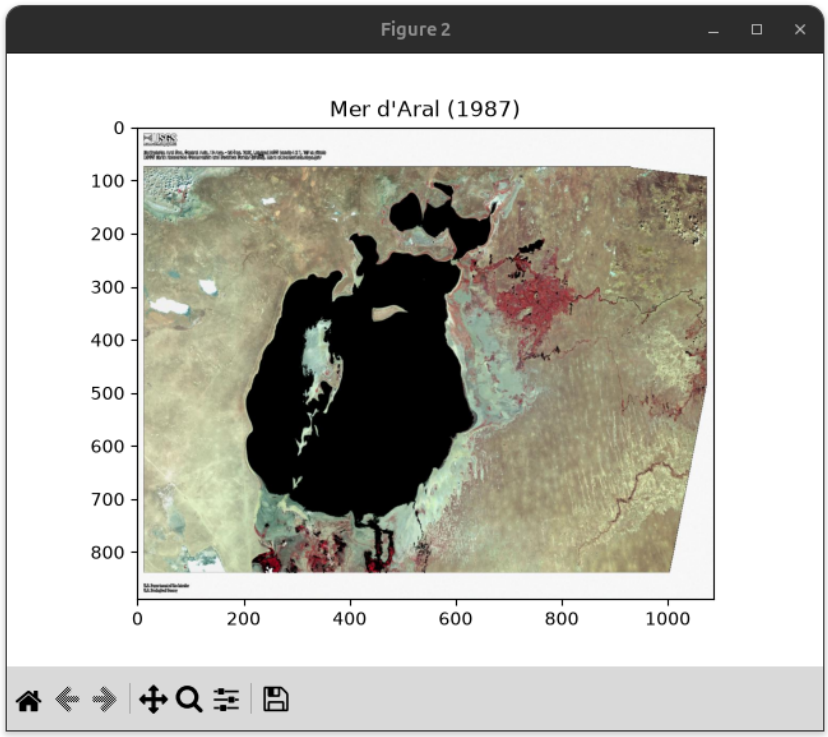
    plt.figure()
    plt.imshow(img73)
    plt.title("Mer d'Aral (1973)")

    plt.figure()
    plt.imshow(img87)
    plt.title("Mer d'Aral (1987)")

    plt.show()

    featLearn = None
    # ------------------------------------------------
    # ------------------------------------------------
    #%% sortie
    return featLearn,img73,img87
```





1.2 | 3.2 Question

R: On observe une diminution de la surface de la mer d'Aral entre 1973 et 1987. Les images sont codées en RGB. Chaque pixel possède trois composantes (rouge,

vert et bleu), avec des valeurs comprises entre 0 et 255.

1.3 | 3.3 Question

R: L'espace des descripteurs est de dimension 3, parce que chaque pixel est représenté par trois composantes: rouge, vert et bleu.

1.4 | 3.4 Question

```python
#%% package
import numpy as np
import matplotlib.pyplot as plt

from loadImages import loadImages
from selectFeatureVectors import selectFeatureVectors
from displayFeatures2d import displayFeatures2d
from displayFeatures3d import displayFeatures3d


#%% def preprocessing
def preprocessing():
    # ------------------------------------------------
    # YOUR CODE HERE
    img73, img87 = loadImages()

    img73 = img73[75:840, :, :]
    img87 = img87[75:840, :, :]

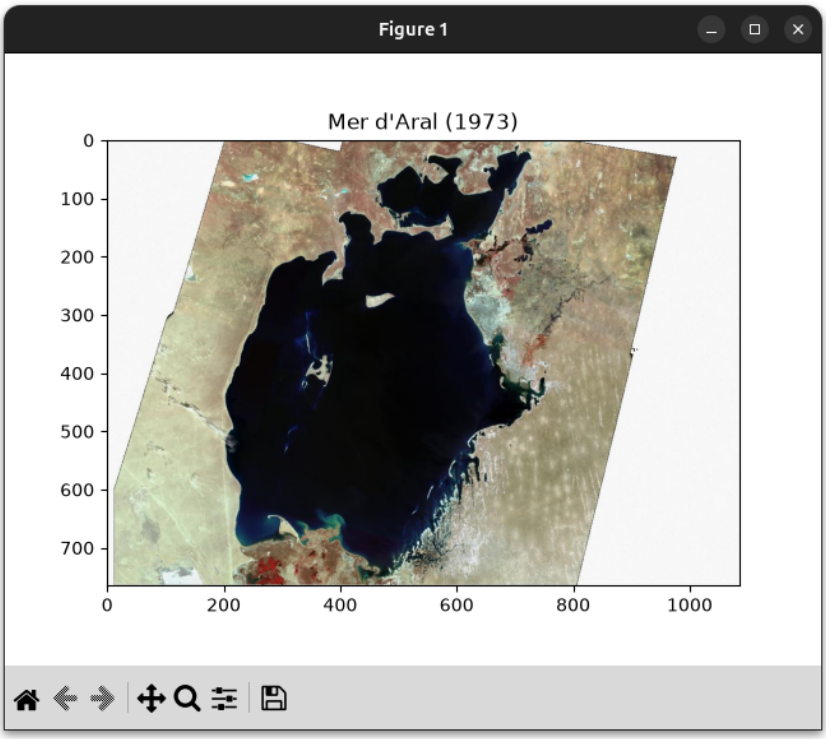
    plt.figure()
    plt.imshow(img73)
    plt.title("Mer d'Aral (1973)")

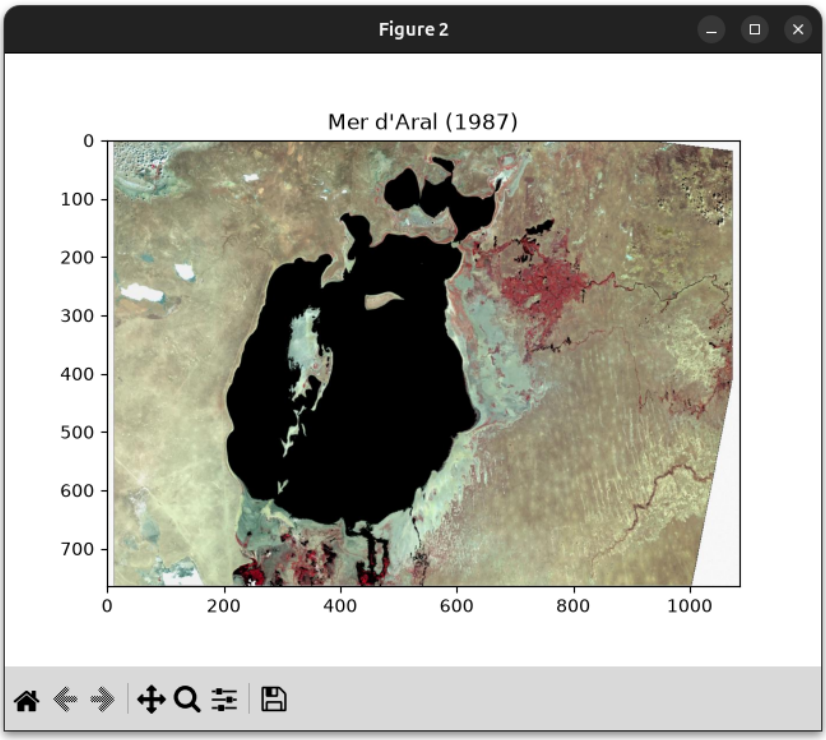
    plt.figure()
    plt.imshow(img87)
    plt.title("Mer d'Aral (1987)")

    plt.show()

    print("Dimensions 1973:", img73.shape)
    print("Dimensions 1987:", img87.shape)

    featLearn = None
    # ------------------------------------------------
    # ------------------------------------------------
    #%% sortie
    return featLearn,img73,img87
```






```
Dimensions 1973: (765, 1086, 3)
Dimensions 1987: (765, 1086, 3)
```

2 | 4 Constitution d'un ensemble de données d'apprentissage

2.1 | 4.1 Question

```python
#%% package
import numpy as np
import matplotlib.pyplot as plt

from loadImages import loadImages
from selectFeatureVectors import selectFeatureVectors
from displayFeatures2d import displayFeatures2d
from displayFeatures3d import displayFeatures3d


#%% def preprocessing
def preprocessing():
    # ------------------------------------------------
    # YOUR CODE HERE
    # Question 3.1
    img73, img87 = loadImages()

    # Question 3.4
    img73 = img73[75:840, :, :]
    img87 = img87[75:840, :, :]

    # Question 4.1
    featLearn, nbPix, nbFeat = selectFeatureVectors(img73,
       500)

    plt.figure()
    plt.imshow(img73)
    plt.title("Mer d'Aral (1973)")

    plt.figure()
    plt.imshow(img87)
    plt.title("Mer d'Aral (1987)")

    plt.show()

    print("Dimensions 1973:", img73.shape)
    print("Dimensions 1987:", img87.shape)

    print("\nNombre de donnees d'apprentissage:", nbPix)
    print("Nombre de descripteurs:", nbFeat)
    print("Dimensions de featLearn:", featLearn.shape)
    # ------------------------------------------------
    # ------------------------------------------------
    #%% sortie
    return featLearn,img73,img87
```

R: On obtient 1662 données d'apprentissage.

```
Nombre de données d'apprentissage: 1662
Nombre de descripteurs: 3
Dimensions de featLearn: (1662, 3)
```

3 | 5 Première analyse des données

3.1 | 5.1 Question

```python
#%% package
import numpy as np
import matplotlib.pyplot as plt

from loadImages import loadImages
from selectFeatureVectors import selectFeatureVectors
from displayFeatures2d import displayFeatures2d
from displayFeatures3d import displayFeatures3d


#%% def preprocessing
def preprocessing():
    # ------------------------------------------------
    # YOUR CODE HERE
    # Question 3.1
    img73, img87 = loadImages()

    # Question 3.4
    img73 = img73[75:840, :, :]
    img87 = img87[75:840, :, :]

    # Question 4.1
    featLearn, nbPix, nbFeat = selectFeatureVectors(img73,
        500)

    # Question 5.1
    displayFeatures2d(featLearn)
    displayFeatures3d(featLearn)

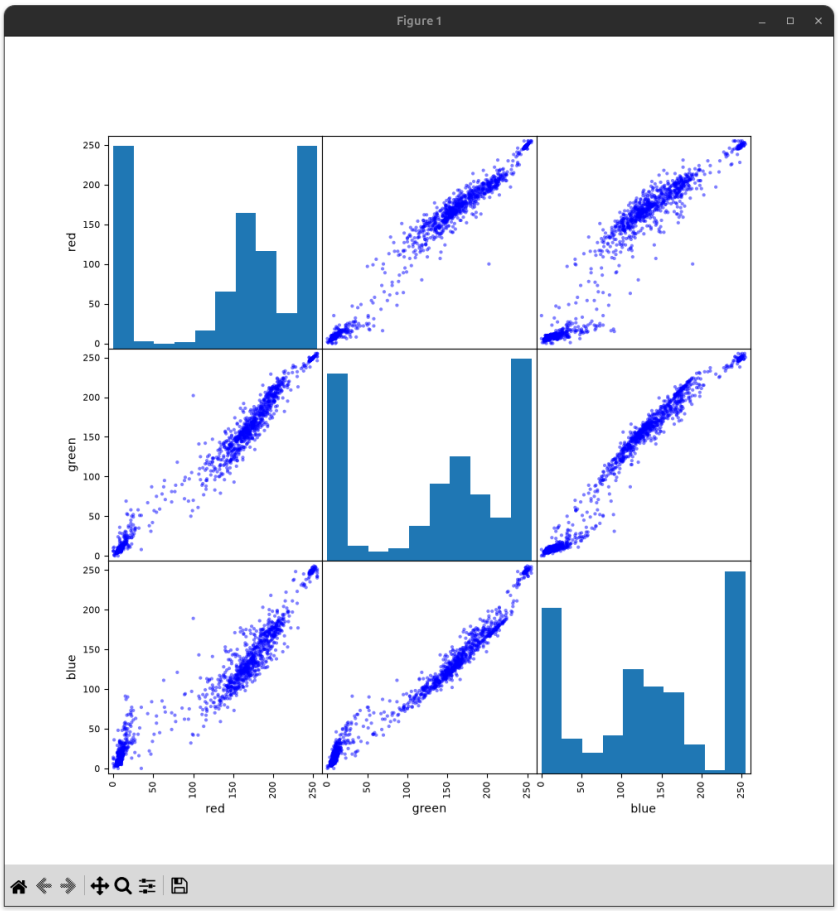
    plt.show()

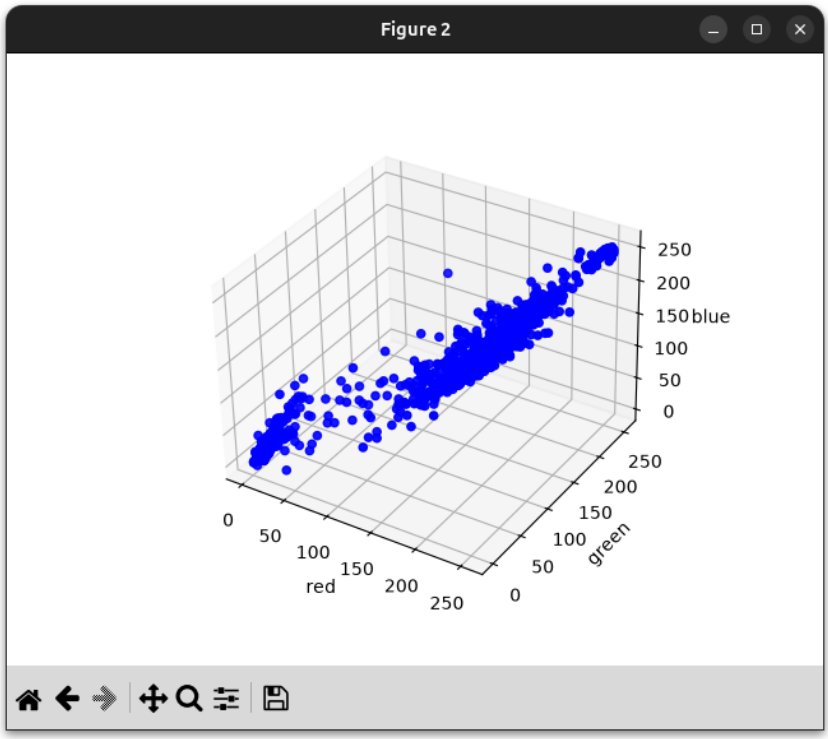
    plt.figure()
    plt.imshow(img73)
    plt.title("Mer d'Aral (1973)")

    plt.figure()
    plt.imshow(img87)
    plt.title("Mer d'Aral (1987)")

```

```
39      plt.show()
40
41      print("Dimensions 1973:", img73.shape)
42      print("Dimensions 1987:", img87.shape)
43
44      print("\nNombre de donnees d'apprentissage:", nbPix)
45      print("Nombre de descripteurs:", nbFeat)
46      print("Dimensions de featLearn:", featLearn.shape)
47      # ------------------------------------------------
48      # ------------------------------------------------
49      #%% sortie
50      return featLearn,img73,img87
```





3.2 | 5.2 Question

R: Les histogrammes montrent la distribution des valeurs des trois composantes RGB. On observe concentrations de valeurs, proches de 0, entre 100 et 200, et proches de 255. On distingue quelques groupes de pixels selon leurs couleurs : les valeurs faibles correspondent principalement à l'eau, les valeurs intermédiaires au sol et à la végétation, et les valeurs élevées aux zones très claires et au fond blanc de l'image.

4 | 6 Approche non supervisée par la méthode des k-means

4.1 | 6.1 Question

R: La méthode des centres mobiles consiste à regrouper les données en K classes selon leurs ressemblances.

On initialise K centres, puis chaque point est associé au centre le plus proche. Ensuite, on recalcule les centres avec la moyenne des points de chaque groupe. On répète ces étapes jusqu'à ce que les centres se stabilisent.

4.2 | 6.2 Question

unsupervisedTraining.py:

```
1 #%% package
```

```python
import numpy as np
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture

#%% def unsupervisedTraining
def unsupervisedTraining(featLearn, method='kmeans'):
    ''' apprentissage avec la fonction KMeans() et
       GaussianMixture
    de scikit-learn :
    - featLearn est la matrice de l'ensemble d'apprentissage
    - method: type d'algorithme de Machine Learning utilise (
       KMeans et GaussianMixture)
    - nbCluster est le nombre de cluster = nombre de classes
       rentre par l'utilisateur en debut de fonction

    - renvoie model: le modele de classement ou classifieur
    '''

    # fixer le nombre de classes
    answer = input('nombre de classes:')
    nbCluster = int(answer)

    if method == 'kmeans':

    # ------------------------------------------------
    # YOUR CODE HERE
        model = KMeans(
            n_clusters=nbCluster,
            init='k-means++',
            n_init=10,
            max_iter=300,
            random_state=42)

        model.fit(featLearn)
    # ------------------------------------------------
    # ------------------------------------------------

    elif method == 'gmm':

    # ------------------------------------------------
    # YOUR CODE HERE
        raise NotImplementedError("GMM pas encore implemente")

    # ------------------------------------------------
    # ------------------------------------------------
    # sortie
    return model
```

aralsea_main.py

```python
#%% Machine Learning Class - Exercise Aral Sea Surface Estimation

# package
import numpy as np
import matplotlib.pyplot as plt

plt.ioff() # to see figure avant input

# ----------------------------------------------------
# YOUR CODE HERE
from preprocessing import preprocessing
from unsupervisedTraining import unsupervisedTraining
# ----------------------------------------------------
# ----------------------------------------------------
#%% Examen des donnees, pretraitements et extraction des descripteurs
# Chargement des donnees et pretraitements
featLearn,img73,img87 = preprocessing()

#%% Apprentissage / Learning / Training
# Apprentissage de la fonction de classement

# ----------------------------------------------------
# YOUR CODE HERE
# Question 6.2
model = unsupervisedTraining(featLearn, method='kmeans')

print("Centres des clusters:", model.cluster_centers_)
print("Nombre d'iterations:", model.n_iter_)
print("Inertie:", model.inertia_)
```

```
nombre de classes:3
Centres des clusters: [[ 13.29052632  15.01263158  20.58315789]
 [168.13916786 160.38020086 133.41606887]
 [244.16326531 244.97346939 242.12653061]]
Nombre d'iterations: 4
Inertie: 2167147.1365185315
```

R: On utilise la fonction ‘KMeans()‘ de scikit-learn pour créer un modèle de classification non supervisée. Ensuite, on utilise ‘fit()‘ pour entraîner le modèle avec les données RGB. Les principaux paramètres sont ‘n_clusters‘, qui définit le nombre de classes, ‘init‘, qui définit l’initialisation des centres, ‘n_init‘, qui définit le nombre d’initialisations, et ‘max_iter‘, qui limite le nombre d’itérations. Le paramètre ‘random_state‘ permet de reproduire les résultats.

4.3 | 6.3 Question

```python
# prediction des labels sur la base d'apprentissage

```

```
# ------------------------------------------------
# YOUR CODE HERE
# Question 6.3
labelLearn = model.predict(featLearn)

print("Dimensions des labels:", labelLearn.shape)
print("Classes identifiees:", np.unique(labelLearn))
```

```
Dimensions des labels: (1662,)
Classes identifiées: [0 1 2]
```

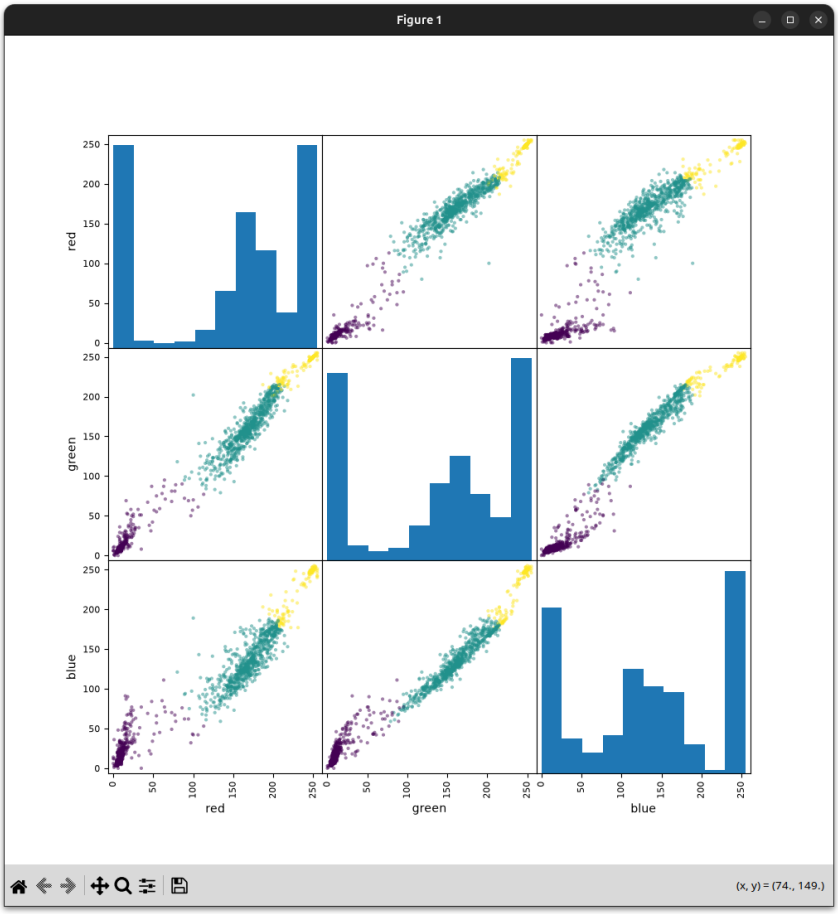
4.4 | 6.4 Question

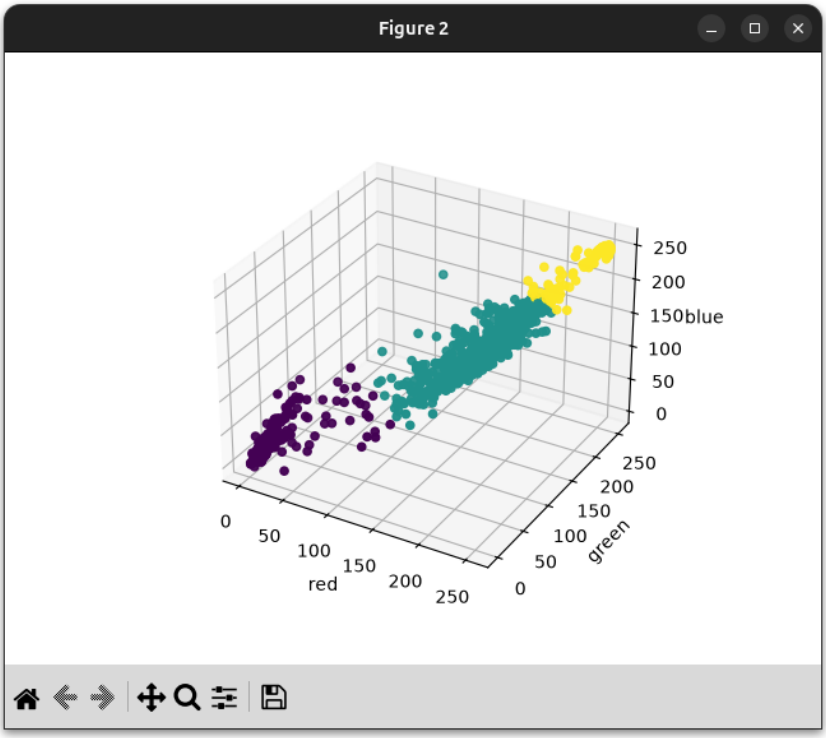
```
# Visualisation des resultats

# ------------------------------------------------
# YOUR CODE HERE
# Question 6.4
displayFeatures2d(featLearn, labelLearn)
displayFeatures3d(featLearn, labelLearn)

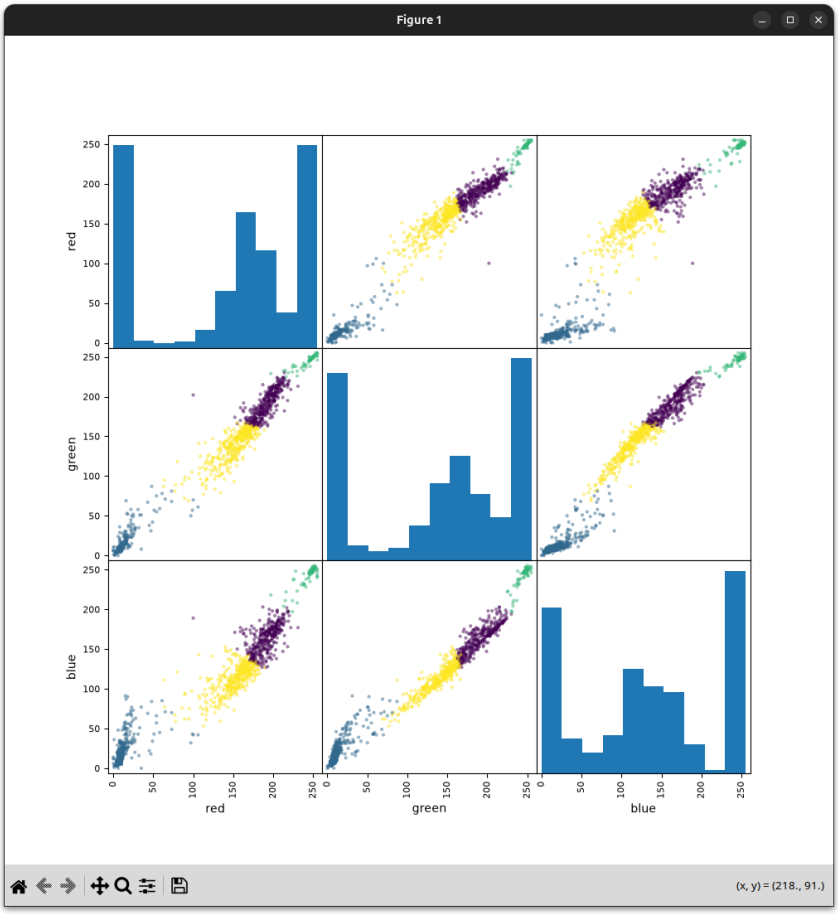
plt.show()
```

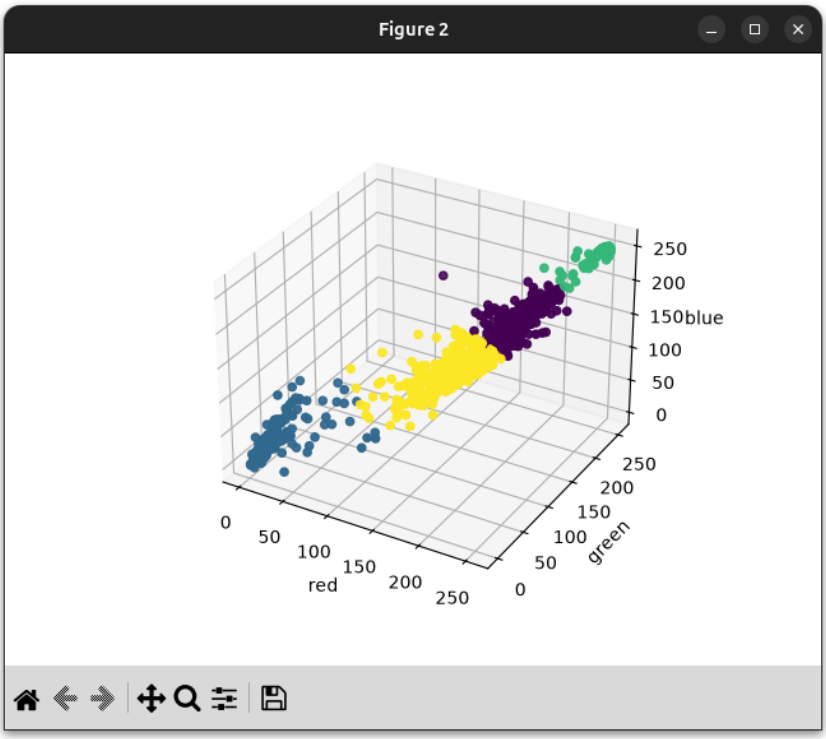
k = 3.


Figure 1
red
green
blue
(x, y) = (74., 149.)

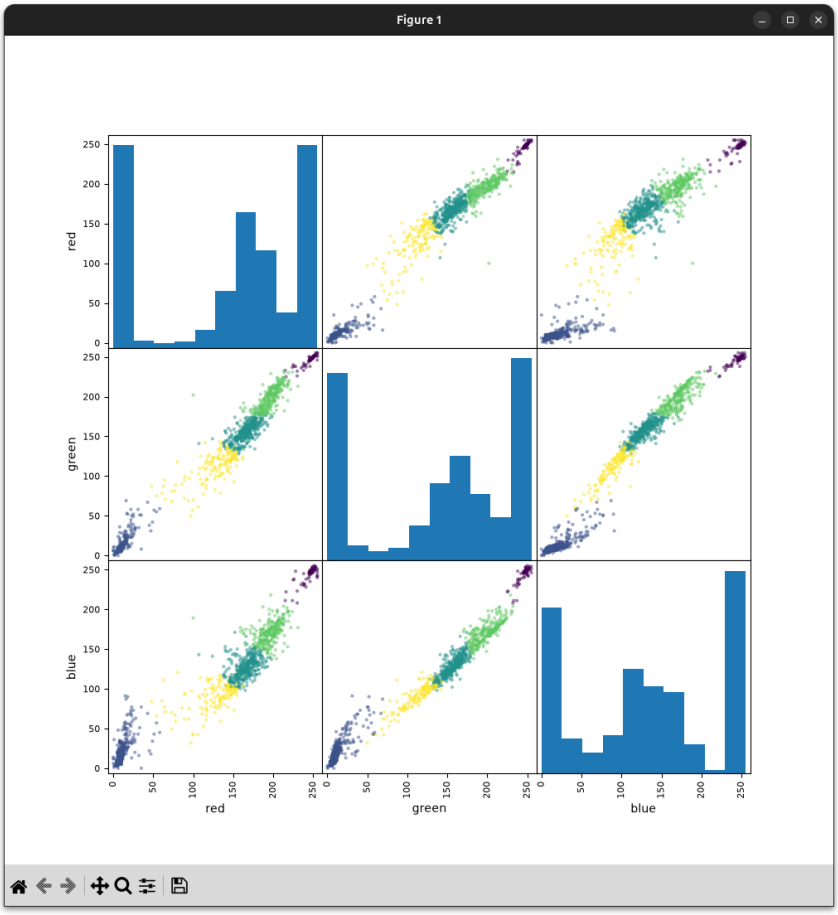

Figure 2
red
green
blue

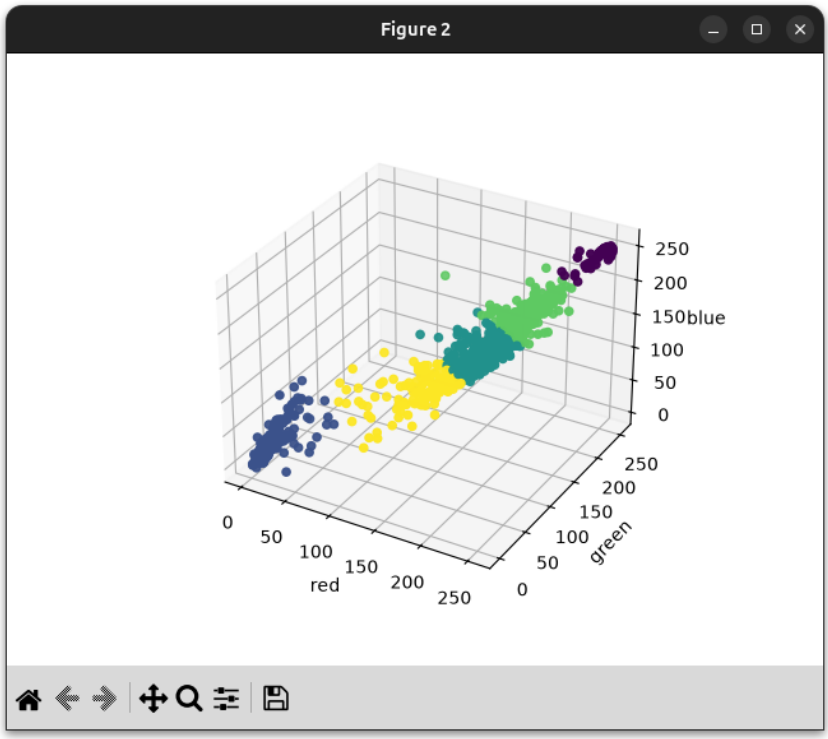
k = 4.




Figure 2
250
200
150blue
100
50
0
0
50
100
150
200
250
red
250
200
150
100
50
0
green

$k = 5$.


Figure 1
red
green
blue
0
50
100
150
200
250



R: On a testé K-means avec 3, 4 et 5 classes. Avec K = 3, on observe trois groupes principaux correspondant aux pixels sombres, intermédiaires et clairs. Quand on augmente le nombre de classes, les groupes sont divisés en plusieurs sous-groupes. Pour notre objectif, K = 3 semble suffisant pour distinguer les pixels de l'eau des autres régions.

4.5 | 6.5 Question

aralsea_main.py:

```
#%% Classement et estimation de la diminution de surface
# Classifying / Predicting / Testing

# mise en forme de l'image de 1973 et 1987 en matrice Num
    Pixels / Val Pixels

# ------------------------------------------------
# YOUR CODE HERE
# Question 6.5
nbPix73 = img73.shape[0] * img73.shape[1]
nbPix87 = img87.shape[0] * img87.shape[1]

feat73 = img73.reshape((nbPix73, 3))
feat87 = img87.reshape((nbPix87, 3))

print("Dimensions des descripteurs 1973:", feat73.shape)
print("Dimensions des descripteurs 1987:", feat87.shape)
```

```
17
18 label73 = unsupervisedClassifying(model, feat73)
19 label87 = unsupervisedClassifying(model, feat87)
20
21 print("Dimensions des labels 1973:", label73.shape)
22 print("Dimensions des labels 1987:", label87.shape)
23
24 print("Classes 1973:", np.unique(label73))
25 print("Classes 1987:", np.unique(label87))
```

unsupervisedClassifying.py:

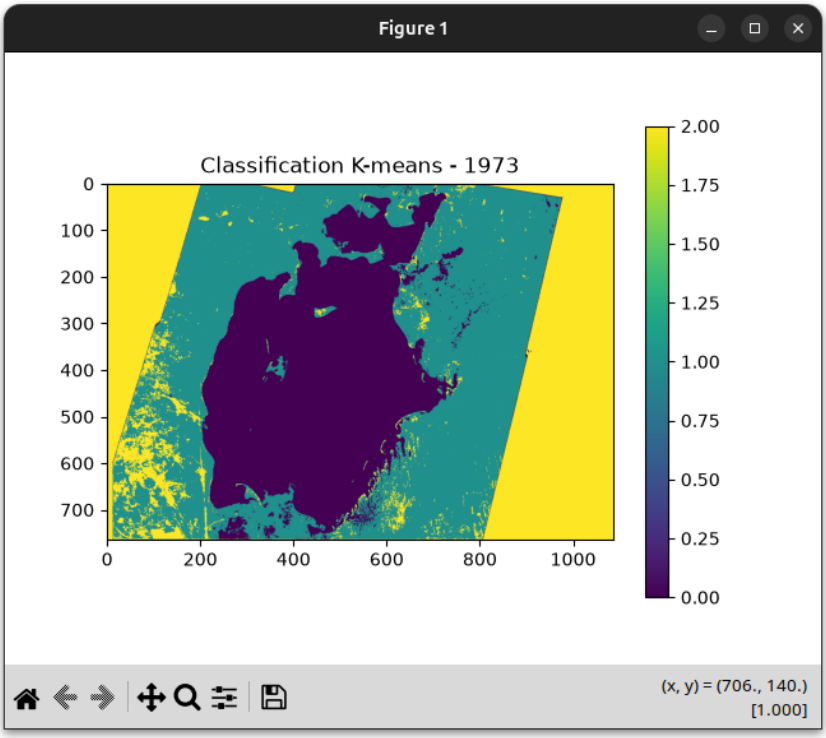
```
1  #%% package
2  import numpy as np
3  from sklearn.cluster import KMeans
4
5
6  #%% def unsupervisedClassifying
7  def unsupervisedClassifying(model, feat):
8      ''' classement/prediction a partir d'un modele de
           classement non supervise
9      feat est la matrice du jeu de donnees a classer
10     label est la classe predite
11     '''
12
13     # ------------------------------------------------
14     # YOUR CODE HERE
15     label = model.predict(feat)
16     # ------------------------------------------------
17     # sortie
18     return label
```

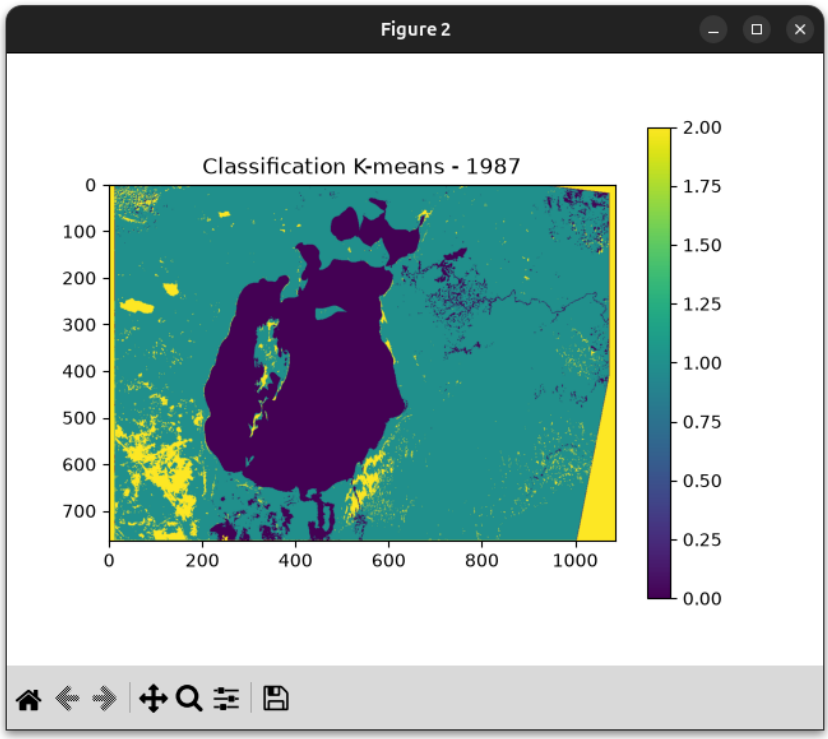
```
Dimensions des descripteurs 1973: (830790, 3)
Dimensions des descripteurs 1987: (830790, 3)
Dimensions des labels 1973: (830790,)
Dimensions des labels 1987: (830790,)
Classes 1973: [0 1 2]
Classes 1987: [0 1 2]
```

4.6 | 6.6 Question

```
1  # Classement des deux jeux de donnees et visualisation des
      resultats en image
2
3  # ------------------------------------------------
4  # YOUR CODE HERE
5  # Question 6.6
6  plt.figure()
7  imgLabel73, imgplot73 = displayImageLabel(label73, img73)
8  plt.title("Classification K-means - 1973")
9
```

```
10 plt.figure()
11 imgLabel87, imgplot87 = displayImageLabel(label87, img87)
12 plt.title("Classification K-means - 1987")
13
14 plt.show()
```





4.7 | 6.7 Question

```python
#%% Estimation de la surface perdue
answer = input('Numero de la classe de la mer ? ')
cl_mer = int(answer)

# ------------------------------------------------
# YOUR CODE HERE
# Question 6.7
surface73 = np.sum(label73 == cl_mer)
surface87 = np.sum(label87 == cl_mer)

diminution = (surface73 - surface87) / surface73 * 100

print("\n--- Estimation de la surface de la mer d'Aral ---")
print("Nombre de pixels en 1973:", surface73)
print("Nombre de pixels en 1987:", surface87)
print(f"Diminution de la surface: {diminution:.2f} %")
```

```
Numero de la classe de la mer ? 0

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 236469
Nombre de pixels en 1987: 182685
Diminution de la surface: 22.74 %
```

R: On a identifié que la classe 0 correspond principalement à l'eau de la mer d'Aral.

On a ensuite compté le nombre de pixels appartenant à cette classe dans les images de 1973 et 1987.

On obtient une diminution d'environ 22,74 % entre 1973 et 1987.

4.8 | 6.8 Question

k = 3.

```
Numero de la classe de la mer ? 0

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 236469
Nombre de pixels en 1987: 182685
Diminution de la surface: 22.74 %
```

k = 4.

```
Numero de la classe de la mer ? 1

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 232639
Nombre de pixels en 1987: 177131
Diminution de la surface: 23.86 %
```

k = 5.

```
Numero de la classe de la mer ? 1

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 229043
Nombre de pixels en 1987: 173261
Diminution de la surface: 24.35 %
```

R: On obtient une diminution de la surface de la mer d'Aral de 22,74 % pour K = 3, 23,86 % pour K = 4 et 24,35 % pour K = 5.

On remarque que le pourcentage estimé augmente légèrement avec le nombre de classes.

5 | 7 Approche non-supervisée par GMM

5.1 | 7.1 Question

R: Le GMM est une méthode d'apprentissage non supervisé qui regroupe les données à partir de plusieurs distributions gaussiennes.

Chaque gaussienne est représentée par une moyenne, une matrice de covariance et un poids. Contrairement au K-means, le GMM calcule la probabilité qu'un point appartienne à chaque classe.

L'apprentissage utilise l'algorithme EM (Expectation-Maximization), qui fonctionne en deux étapes :

E-step : calculer les probabilités d'appartenance de chaque point aux différentes gaussiennes.

M-step : mettre à jour les moyennes, les covariances et les poids à partir de ces probabilités.

Ces deux étapes sont répétées jusqu'à la convergence du modèle. Ensuite, chaque pixel est classé dans le groupe ayant la probabilité la plus élevée.

5.2 | 7.2 Question

unsupervisedTraining.py:

```
#%% package
import numpy as np
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture

#%% def unsupervisedTraining
def unsupervisedTraining(featLearn, method='kmeans'):
    ''' apprentissage avec la fonction KMeans() et
       GaussianMixture
    de scikit-learn :
    - featLearn est la matrice de l'ensemble d'apprentissage
    - method: type d'algorithme de Machine Learning utilise (
       KMeans et GaussianMixture)
    - nbCluster est le nombre de cluster = nombre de classes
       rentre par l'utilisateur en debut de fonction

    - renvoie model: le modele de classement ou classifieur
    '''

    # fixer le nombre de classes
    answer = input('nombre de classes:')
    nbCluster = int(answer)

    if method == 'kmeans':

    # ------------------------------------------------
    # YOUR CODE HERE
        model = KMeans(
            n_clusters=nbCluster,
            init='k-means++',
            n_init=10,
            max_iter=300,
            random_state=42)

        model.fit(featLearn)
    # ------------------------------------------------
    # ------------------------------------------------

    elif method == 'gmm':

    # ------------------------------------------------
    # YOUR CODE HERE
        # Question 7.2
        model = GaussianMixture(
            n_components=nbCluster,
            covariance_type='full',
```

```
            n_init=10,
            max_iter=300,
            random_state=42)

        model.fit(featLearn)

    # -----------------------------------------------
    # -----------------------------------------------
    # sortie
    return model
```

aralsea_main.py:

```
#%% Apprentissage / Learning / Training
# Apprentissage de la fonction de classement

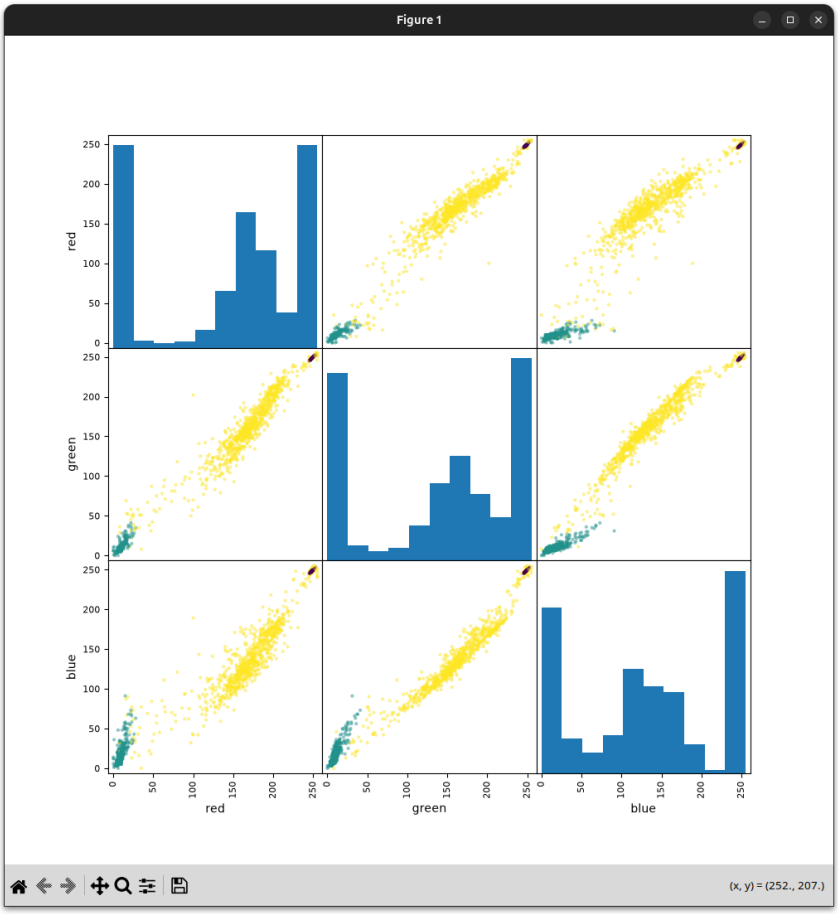
# -----------------------------------------------
# YOUR CODE HERE
# Question 6.2 / 7.2
# model = unsupervisedTraining(featLearn, method='kmeans')
model = unsupervisedTraining(featLearn, method='gmm')

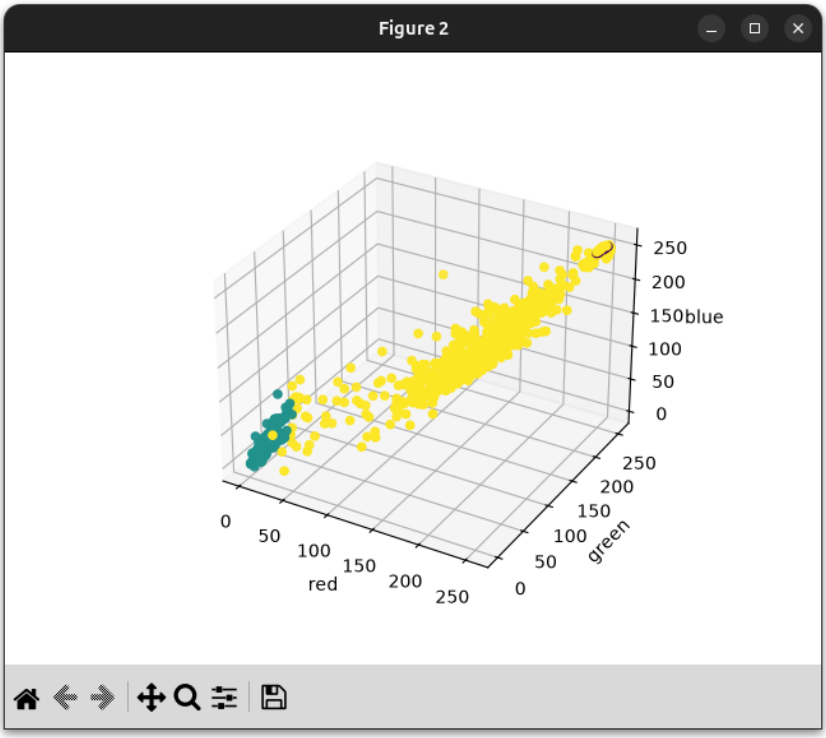
# print("Centres des clusters:", model.cluster_centers_)
# print("Nombre d'iterations:", model.n_iter_)
# print("Inertie:", model.inertia_)
print("Moyennes des gaussiennes:")
print(model.means_)

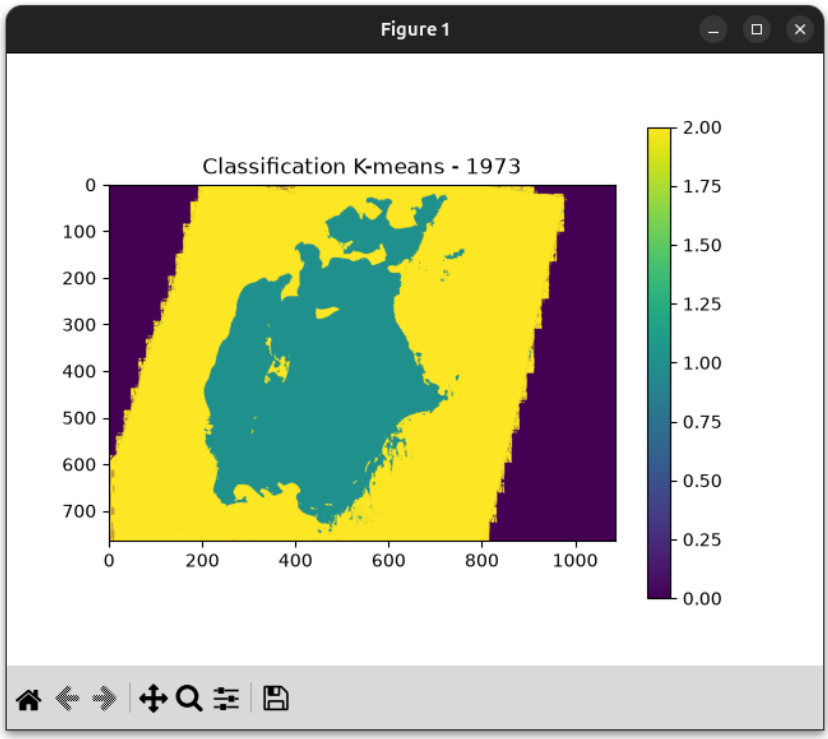
print("Poids des gaussiennes:")
print(model.weights_)

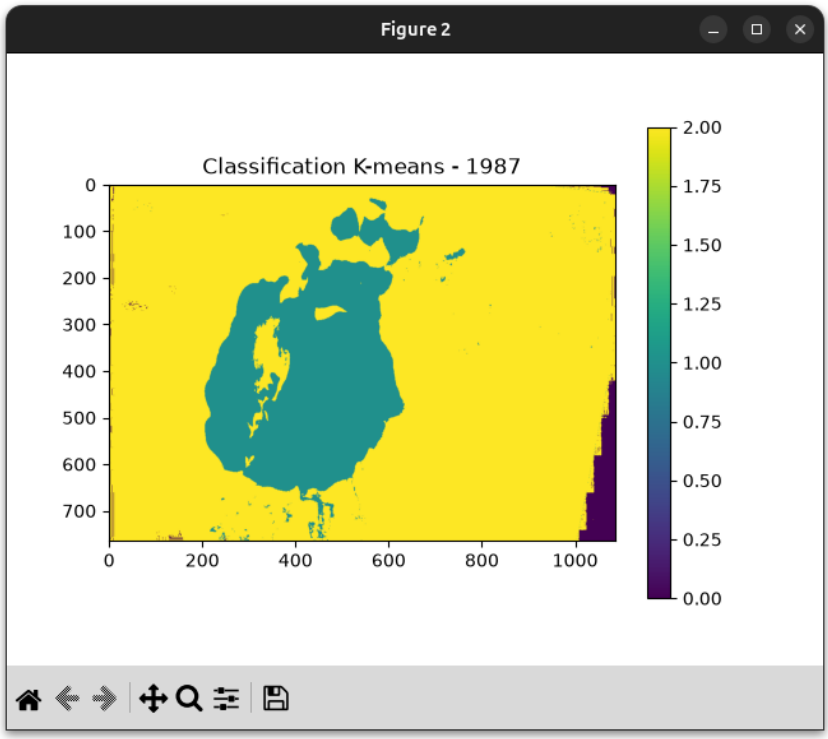
print("Nombre d'iterations:", model.n_iter_)
print("Convergence:", model.converged_)
```

INFORMATION IMPORTANTE: J'ai oublié de changer le titres des figures pour le question 7.4, donc elles sont encore intitulées "Classification K-means".


Figure 1
red
green
blue
(x, y) = (252., 207.)


Figure 2
red
green
blue






```
Numero de la classe de la mer ? 1

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 213579
Nombre de pixels en 1987: 163338
Diminution de la surface: 23.52 %
```

5.3 | 7.3 Question

k = 3, full.

```
Numero de la classe de la mer ? 1

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 213579
Nombre de pixels en 1987: 163338
Diminution de la surface: 23.52 %
```

k = 4, full

```
Numero de la classe de la mer ? 0

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 183762
Nombre de pixels en 1987: 159119
Diminution de la surface: 13.41 %
```

k = 5, full.

```
Numero de la classe de la mer ? 3

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 183295
Nombre de pixels en 1987: 159027
Diminution de la surface: 13.24 %
```

k = 3, diag.

```
Numero de la classe de la mer ? 0

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 216074
Nombre de pixels en 1987: 166563
Diminution de la surface: 22.91 %
```

k = 3, spherical.

```
Numero de la classe de la mer ? 0

--- Estimation de la surface de la mer d'Aral ---
Nombre de pixels en 1973: 216557
Nombre de pixels en 1987: 168431
Diminution de la surface: 22.22 %
```

R: On a étudié l'influence des paramètres du GMM sur la classification des pixels.

Avec une covariance de type 'full', on obtient une diminution estimée de 23,52 % pour K = 3, 13,41 % pour K = 4 et 13,24 % pour K = 5. On remarque donc que l'augmentation du nombre de composantes modifie fortement la répartition des pixels entre les classes.

Avec K = 3, on a aussi testé d'autres types de covariance. On obtient 22,91 % avec 'diag' et 22,22 % avec 'spherical'. Ces résultats restent proches de celui obtenu avec 'full'.

On peut conclure que le GMM est sensible au choix des paramètres, en particulier au nombre de composantes. La configuration K = 3 donne ici les résultats les plus cohérents.

5.4 | 7.4 Question

R: Les deux méthodes permettent de classer les pixels des images et d'estimer l'évolution de la surface de la mer d'Aral.

Le K-means donne des résultats stables, avec une diminution estimée comprise entre 22,74 % et 24,35 % selon le nombre de classes. La zone de l'eau est aussi bien identifiée sur les images classifiées.

Le GMM donne de bons résultats avec 3 composantes, par exemple 23,52 % avec une covariance 'full', 22,91 % avec 'diag' et 22,22 % avec 'spherical'. En revanche, les résultats changent beaucoup quand on augmente le nombre de composantes, avec seulement 13,41 % pour K = 4 et 13,24 % pour K = 5.

On peut donc conclure que, dans ce TP, le K-means fournit le résultat le plus robuste et le plus simple à interpréter.